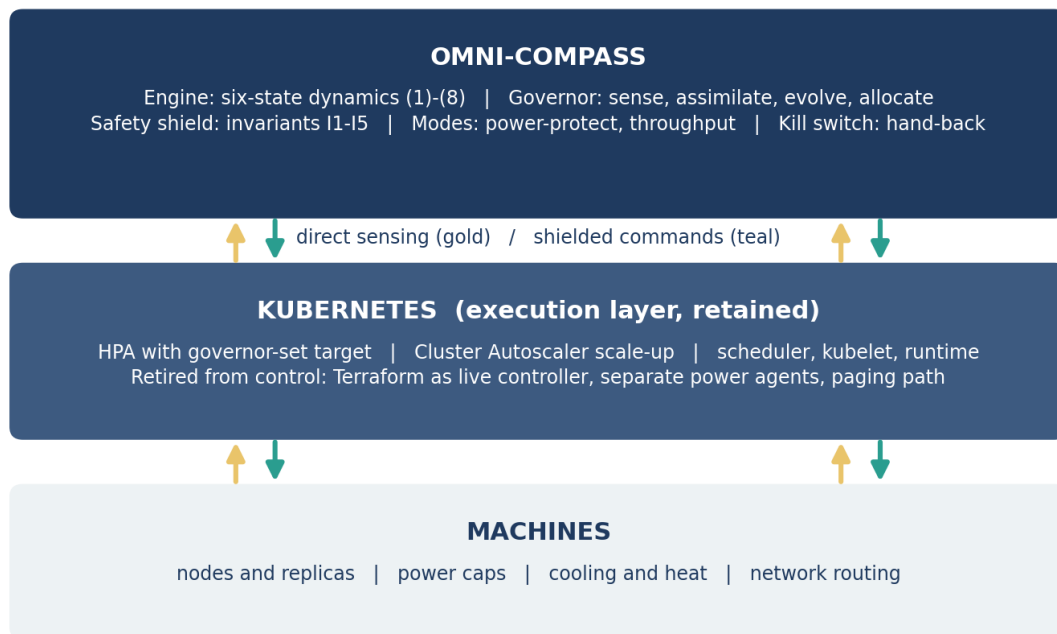


OMNI-COMPASS

A six-state control engine governing Kubernetes

Technical Manual and Referee Record



The Omni-Compass LLC

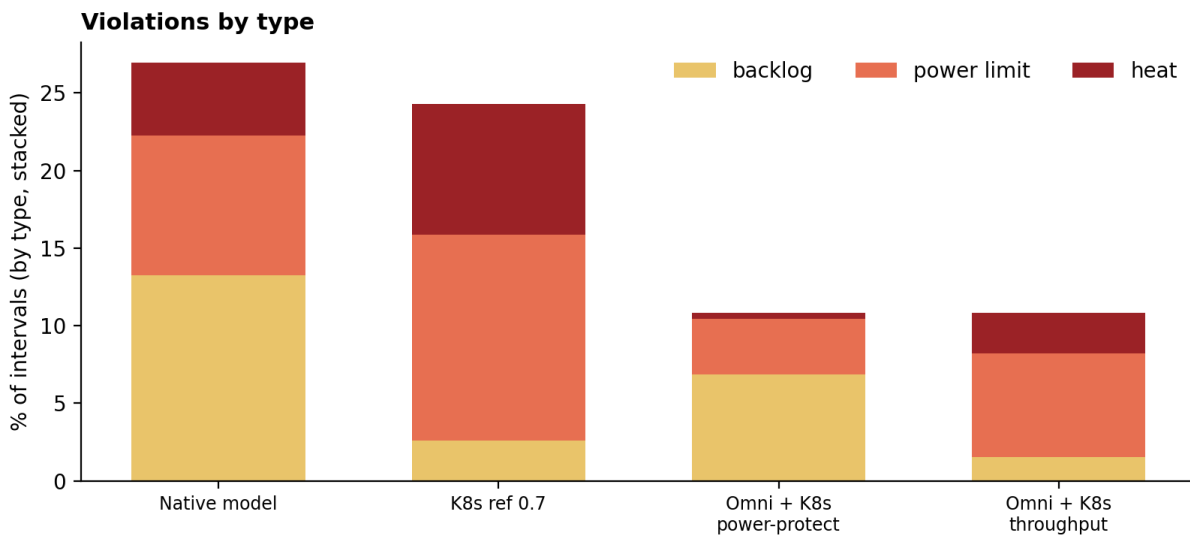
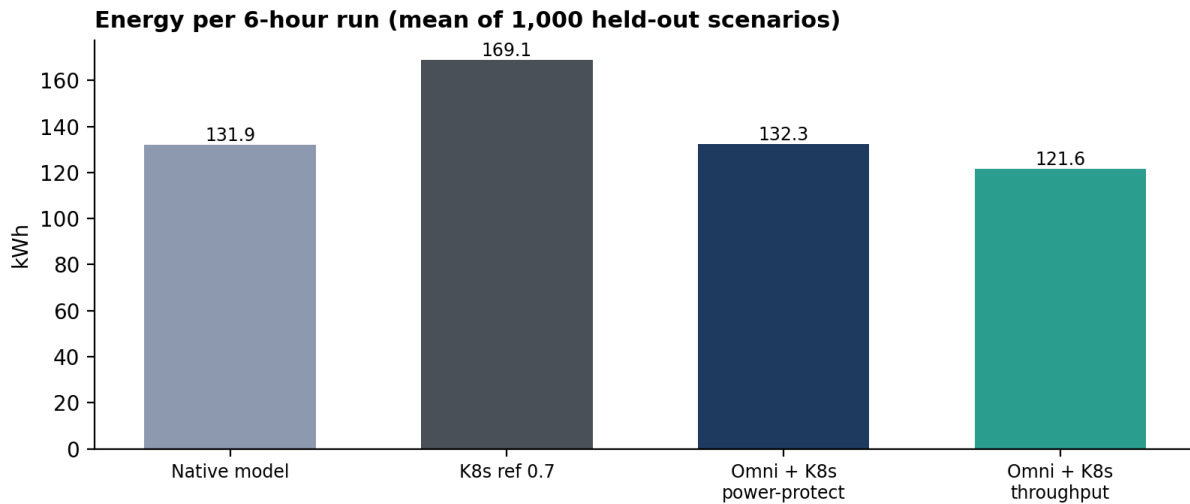
This manual describes only what the accompanying package contains and executes. Every number and chart is generated from the result files in `results/` by the build scripts. Every claim is paired with the file and command that establish it. Anything not established by the package is listed in Chapter 10.

Executive Summary

Omni-Compass is a control engine that sits above Kubernetes. Kubernetes remains the execution layer: it schedules and runs the containers, and its autoscalers react to unscheduled work. Omni-Compass is the single authority above it: it reads the machines directly, evolves a six-state dynamical model of the stack, sets Kubernetes' scaling targets, owns scale-down, power caps, rollback authority and routing, and passes every command through a safety shield before execution. Components whose only role was to make competing control decisions (Terraform as a live capacity controller, separate power agents, the paging path) are retired from the control loop. The engine runs in two operator-selected modes: power-protect, which enforces the site power limit, and throughput, which applies the same power rules as the native stack.

Benefit	Kubernetes reference (HPA 0.7)	Omni + K8s power-protect	Omni + K8s throughput	Evidence
Energy per run	169.1 kWh	132.3 kWh (-22%)	121.6 kWh (-28%)	Simulation, 1,000 held-out
Time healthy	78.9%	89.8%	91.3%	Simulation
Recovery after events	58 min	28 min	17 min	Simulation
Physical violations (backlog + power + heat)	14.4%	8.9%	7.3%	Simulation
Backlog violations	2.6%	6.8%	1.5%	Simulation
Power / heat violations	13.3% / 8.4%	3.6% / 0.4%	6.7% / 2.6%	Simulation
Contradictory commands per run	9.5	0.0	0.0	Simulation
Invariant violations per run, I1-I5	9.50	0.00	2.71	Simulation
Invariant violations per run, excluding I4 (power)	6.86	0.00	0.00	Simulation
Thermal cycling (hardware wear)	0.75	0.59	0.63	Simulation
Scale direction reversals (wear)	2.20	3.07	1.93	Simulation
Machines started / stopped per run	4.8 / 1.2	10.7 / 2.6	6.6 / 3.3	Simulation
Machine round trips (started, later stopped)	1.1	2.6	1.7	Simulation
Engine cost per decision	n/a	2.6 microseconds	2.6 microseconds	Measured, C++, test machine
Continuous operation	n/a	100,000,000 decisions, 0 failures	same engine	Measured, C++ soak

Means over held-out seeds 346410161 and 360555127 (500 scenarios each). Kubernetes reference = documented-behaviour model with HPA target 0.7; results against targets 0.5, 0.6 and 0.8 are in Section 8.3. The throughput mode is the like-for-like comparison, because it applies the same power rules as the reference; power-protect additionally enforces the site power limit, which the reference does not. Throughput mode does not enforce invariant I4 (projected power), so its I4 count is reported rather than excluded. Machine start/stop The reference model removes a machine only after 10 minutes below 50% utilization, so it keeps surplus and post-event machines running; the governor stops them, which is the main source of its energy saving. Round trips count machines started and later stopped (or the reverse); scale direction reversals measure back-and-forth. Hardware wear is also measured by thermal travel.



Against a 15-second Kubernetes control plane (fleet harness, Section 8.8, synthetic workloads), with the governor as the node-pool authority in place of the Cluster Autoscaler (HPA retained), energy changes by: web -34% vs CA, -17% vs Karpenter-lite; multi -34% vs CA, -17% vs Karpenter-lite; batch -17% vs CA, -10% vs Karpenter-lite; gpu -5% vs CA, -3% vs Karpenter-lite; gpu_always_on -5% vs CA, -3% vs Karpenter-lite. It reverses node direction less often than Karpenter-lite in every vessel and more often than the Cluster Autoscaler. The results above this paragraph use the 5-minute stack model and its documented-behaviour reference, which is slower than the 15-second control plane; the fleet harness is the stronger comparison.

Operational burden. The governor holds every lever a paged operator would use for capacity, power and overload events and issues no page actions; engineer hours are a pilot measurement (docs/PILOT_PROTOCOL.md). **What is not claimed.** Production performance, behaviour of the upstream Kubernetes controllers, and cybersecurity beyond the stated shield invariants are not established by this package (Chapter 10).

1. What Is Established

Claim	Status	Evidence
The six-state mechanism (equations 1 to 8) is implemented identically in the reference engine, the Python core and the C++ core.	Established	tests/test_core_parity.py; C++ fixture comparison
The C++ stack governor reproduces the Python governor exactly.	Established	tests/test_cpp_governor_parity.py
The governor ran 100,000,000 consecutive decisions with 0 failures and constant memory.	Established	results/SOAK.json
Observe mode leaves the stack bit-identical to the native stack.	Established in simulation	500/500 and 500/500 held-out scenarios
Observe mode over Kubernetes leaves Kubernetes bit-identical to Kubernetes alone.	Established in simulation	Section 8.5
Executed actions in power-protect mode satisfy invariants I1 to I5; throughput mode satisfies I1 to I3 and I5.	Established in simulation	Chapter 8
Stack outcomes of Omni-Compass governing Kubernetes relative to a documented-behaviour reference model of Kubernetes autoscaling, including costs.	Established for the declared synthetic model	Chapter 8
Behaviour on production systems.	Not established	docs/PILOT_PROTOCOL.md

Companion documents: docs/CLAIMS_REGISTER.md (every claim with its status and reproduction command), docs/DUE_DILIGENCE.md, docs/PILOT_PROTOCOL.md.

2. The Mechanism

State $x = (E, U, I_U, S, B, B\blacksquare)$. The engine evolves the following law. Source: `omnicompass/core.py`, `cpp/src/core.cpp`.

```

(1) dE/dt    = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt    = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u ,    |u| <= 25
(3) dI_U/dt  = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff    = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S)   = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt    = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt    = B_dot ,    dB_dot/dt = gamma_c delta S - (omega_B/Q_B) B_dot - omega_B^2 B
(8) R_B[n]   = (B[n+1]-2B[n]+B[n-1])/h^2 + (omega_B/Q_B) (B[n]-B[n-1])/h
              + omega_B^2 B[n] - gamma_c delta S[n]
```

Equation (8) is an audit: it is evaluated on the generated trajectory and never fed back into the state. Equation (5) is the potential whose negative gradient is (6). The factor $\cos(\omega_B t/2)$ returns to +1 after a phase advance of 4π (720 degrees); this is a property of the implemented modulation and carries no physical interpretation in this package.

2.1 Parameters

Parameter	Range	Role
alpha_E	4.2	E decay
beta_int, beta_ext	0 to 1	E forcing
sigma_1	0.38	E coupling into I_U
delta	0.01 to 1	S source; S coupling into I_U and bath

Parameter	Range	Role
gamma_c	1	bath forcing gain
lambda_0	-2 to 2	drive bias
lambda_1	0.1 to 3	U gain in drive
lambda_2	0 to 2	S gain in drive
c	0.5 to 5	drive amplitude
E_max	1 to 10	E-rate normalisation in (2)
omega_B	0.1 to 5	bath frequency and phase rate
Q_B	0.5 to 10	bath quality factor
alpha_s, beta_s	0.05 to 0.25	S potential coefficients
mu	2 to 6	double-well coefficient
lambda_I	0.5 to 2	I_U damping
lambda_U	0 to 0.5	U damping

The record fields alpha, k, gamma_1 and alpha_U are carried for compatibility with the reference engine and do not enter equations (1) to (8).

2.2 Controller

Each macro interval (0.1) is divided into 10 micro steps ($h = 0.01$). At the start of each micro step:

```

u = clip( -f_U(x, t) + KP (sigma - U) , -25, +25 ),    KP = 12
f_U(x, t) = right-hand side of (2) with u = 0
sigma      = +1 if |U0 - 1| <= |U0 + 1| else -1    (locked for the trajectory)

```

u is held constant through all four stages of the classical RK4 step. The accepted state is the RK4 result; no state component is overwritten after integration.

2.3 Event predicates

A value U is in a basin if $|U - 1| \leq 0.10$ or $|U + 1| \leq 0.10$. CONVEY-5 is five consecutive macro observations in the same basin; CERT-10 is ten. Step 0 counts as an observation when U0 is already in a basin. These are internal predicates of the simulation, not external certifications.

3. Mathematical Properties

Proposition 1 (S channel). For $\alpha_s, \beta_s, \delta > 0$, equation (6) has roots $S_- = (-\alpha_s - \sqrt{\alpha_s^2 + 3\beta_s\delta}) / (1.5\beta_s) < 0 < S_+ = (-\alpha_s + \sqrt{\alpha_s^2 + 3\beta_s\delta}) / (1.5\beta_s)$. The interval $[S_-, \infty)$ is forward invariant, and every trajectory with $S_0 > S_-$ converges to S_+ .

Proof. The right-hand side of (6) is the concave quadratic $g(S) = \delta - \alpha_s S - (3/4)\beta_s S^2$, positive exactly on (S_-, S_+) and negative outside. $g(0) = \delta > 0$ places the roots on either side of 0. At $S = S_-$ the field vanishes, so no solution crosses it (uniqueness, since g is smooth). On (S_-, S_+) S increases and on (S_+, ∞) it decreases, each bounded monotonically by S_+ ; hence convergence. ■

On the canonical population all 500 initial states satisfy $S_0 \geq S_-(p)$ (minimum margin 0.191). Over the full declared box, $S_0 = -1$ can lie below S_- for some parameters, in which case S diverges downward; admissibility is therefore a per-trajectory condition, not a property of the whole box.

Proposition 2 (double well). The autonomous term $\mu U(1 - U^2) = -dW/dU$ with $W = (\mu/4)(U^2 - 1)^2$. For $\mu > 0$ its equilibria are $U = -1, 0, +1$; the linearisation is $\mu(1 - 3U^2)$, equal to -2μ at $U = \pm 1$ (stable) and $+\mu$ at $U = 0$ (unstable). This classifies one term of (2), not the coupled system.

Proposition 3 (what the controller guarantees). If the command is applied continuously and is not saturated, $u(t) = -f_U(x,t) + KP(\sigma - U)$ gives $dU/dt = f_U + u = KP(\sigma - U)$ exactly, so $U(t) = \sigma + (U_0 - \sigma)e^{-KP t}$, independently of every other state and parameter.

Consequence: under the canonical controller, arrival of U in the target basin is a property of the controller, not of the dynamics (1) to (7). The following counterfactuals, all on the identical 500-member population (seed 223387268), quantify this. Source: `benchmarks/core_evidence.py`.

Variant	CONVEY-5	CERT-10	Peak $ u $	Saturated micro steps
Canonical controller	500/500	500/500	13.93	0
Controller aimed at the opposite basin	500/500	500/500	25.00	5
Canonical controller, $\mu = 0$	500/500	500/500	14.31	0
No control ($u = 0$)	476/500	427/500	0	0

The closed-form $U(t)$ of Proposition 3 predicts the exact CONVEY-5 confirmation step in 492/500 canonical runs. The 500/500 result is therefore correctly read as: the implemented controller places U in the chosen basin within 0.4 to 0.6 time units on every sampled trajectory, without saturating. It is not evidence about equations (1), (3) to (7), and it is not a stability theorem for the coupled system.

4. Numerical Method

Classical fourth-order Runge-Kutta, $h = 0.01$, command held constant over each step (zero-order hold). With u fixed, RK4 is fourth-order accurate for the plant. The closed loop is a sampled-data system: the held command is itself an $O(h)$ approximation of continuous feedback, so the loop converges at first order in h . Measured on run 0 of the canonical population (maximum state difference at $t = 2$ against $h = 0.000625$):

h	0.02	0.01	0.005	0.0025
$\max x_h - x_{ref} $	1.02e-03	4.90e-04	2.27e-04	9.73e-05

The error halves as h halves, which is first order. The canonical results are the defined sampled-data system at $h = 0.01$; they are not claimed as a converged approximation of a continuous-time controller.

Bath audit (8): median RMS residual 4.91e-03, maximum 7.65e-02 over 100 controlled trajectories. The residual measures consistency of the generated bath trajectory with (7) under a first-order backward difference; it is not an independent physical validation.

5. Implementations, Parity and Endurance

Artifact	Test	Result
omnicompass/core.py vs reference engine functions	20,000 random (state, parameter, time, command) cases	$\max diff \leq 1e-12$
omnicompass/core.py vs frozen fixtures	500 full trajectories, 41 output fields each	0 mismatches
cpp core vs frozen fixtures	500 full trajectories	0 mismatches
cpp governor vs Python governor	recorded telemetry, 60 scenarios x 72 intervals	0 discrete mismatches, $\max diff \leq 1e-12$

Artifact	Test	Result
negative control	mutated C++ governor (dwell 3 instead of 5)	rejected by the parity test
cpp shield vs Python shield	action sets recorded from shielded, unshielded, power-protect, throughput and reference arms	0 mismatches in enforced actions, interventions and violations
negative control	C++ shield no longer blocking rollouts during a security block	rejected by the parity test
independent C++ HPA vs fleet harness HPA	every HPA step of 3 Kubernetes arms and 2 governor arms, web and multi-cluster vessels	0 mismatches in replicas
negative control	C++ HPA with 5% instead of 10% tolerance	rejected by the parity test
omnicompass/stack_sim.py vs reference engine	source extraction	11/11 blocks byte-identical
native benchmark arm vs reference simulator	60 scenarios	identical outputs

The reference engine `reference/omni_compass_reference_engine.py` is the authority for the mechanism. It is the source engine (SHA-256 `c527df2d3d217a62...`) with comments and docstrings removed; its program fingerprint (`cd4e6eb22cd50e14...`) is identical to the source engine's under Python 3.12. Program fingerprints use `ast.dump` and are interpreter-version dependent; SHA-256 hashes are authoritative. Loading the reference engine writes runtime files next to it, so the tests load it from a temporary copy.

5.1 Endurance

The C++ governor executed 100,000,000 consecutive decisions (equivalent to 951 years at 5-minute intervals) on randomized observations spanning and exceeding the sensor ranges, feeding its own power cap and node count back as inputs. Non-finite or out-of-range outputs: 0. Resident memory: 3532 kB at start, 3988 kB at end. Time per decision: 2.55 microseconds. Observed state ranges: E [0.013, 1.026], U [0.452, 1.017], I_U [0.287, 1.659], S [-0.008, 1.134], B [0.067, 1.463]. Resident memory at 25, 50, 75 and 100% of the run: 3852, 3852, 3852, 3852 kB (power-protect); 3916, 3916, 3916, 3916 kB (throughput, 100,000,000 decisions, 0 failures). The initial rise is start-up allocation; memory is flat thereafter.

With zero failures in $n = 100,000,000$ decisions, the rule of three bounds the per-decision failure rate below $3/n = 3.0e-08$ at 95% confidence, under the tested input distribution. Every observation is clamped and assimilation is a convex combination, so assimilated states are bounded; a uniform-ultimate-boundedness theorem for the full sampled-data loop is an open obligation (Chapter 10). The Python governor keeps a fixed-length decision log (4,096 entries). The full verifier reruns the 100,000,000-decision soak and checks that decisions, failures and state ranges match this record.

6. Architecture and Stack Governor

Layer	Content	Files
Engine	Equations (1) to (8), controller, predicates; verified and frozen	<code>omnicompass/core.py</code> , <code>cpp/src/core.cpp</code>
Connector (vessel: Kubernetes stack)	Sensing, assimilation, evolution, allocation law, authority	<code>omnicompass/adaptor.py</code> , <code>cpp/src/governor.cpp</code>
Features	Safety shield; fleet component model	<code>omnicompass/shield.py</code> , <code>omnicompass/muscles.py</code>

6.1 Sense and assimilate

Each 5-minute interval the governor reads queue ratio q , load, power stress, thermal, network stress, configuration drift, staleness, security block and the count of conflicting manager proposals. With direct sensing these are the current values; with pipeline sensing they are delayed by the scenario's telemetry and Kafka lag.

```

E_obs = 0.25q + 0.18max(0, load-0.85) + 0.16power + 0.13thermal + 0.10network + 0.08drift + 0.06conflict
U_obs = 1 - (0.27q + 0.18power + 0.16thermal + 0.12network + 0.12drift + 0.10conflict + 0.05stale)
S_obs = 0.36thermal + 0.28power + 0.18network + 0.10security + 0.08stale - 0.20q
B_obs = 0.52power + 0.26thermal + 0.22drift          I_obs = q + drift + conflict
x      <- 0.661 x + 0.339 obs      (per component; B_dot <- 0.661 B_dot + 0.339 (B_obs - B))
beta_int = 0.58q + 0.42max(0, load-0.75)      beta_ext = 0.33power + 0.27thermal + 0.20network + 0.12

```

The state is then advanced one macro interval by equations (1) to (7) with $u = 0$. Engine parameters used by the governor: $\mu = 3.782$, $\alpha_E = 3.487$, $\lambda_I = 0.719$, $\delta = 0.705$, $\text{assimilation} = 0.339$; the remaining parameters are listed in results/PREREGISTRATION.json.

6.2 Allocation law

```

headroom      rho* = clamp(rho0 - kI I_U - kE E, rho_min, rho0)
capacity      n_req = ceil(n load / rho*) + ceil(kq q n); delta = clamp(n_req - n, -down_max, +up_max)
gate          no removal unless the equation (2) push u/U_AUTHORITY <= push_release, where
              u = clip(-f_U(x) + KP (1 - U), -25, 25) on the evolved engine state (target sigma = +1)
guard         no removal while q >= guard_queue, or while n_req > n - down_band
dwell         removal only after down_dwell consecutive surplus intervals and >= down_after_add interval
envelope      no addition while power >= power_ceiling (power_ceiling_backlog while q >= guard_queue)
security      no addition while a security block is observed
power cap     q < guard_queue and U >= U_gate : cap = clamp(cap_now load (1 + margin), cap_min, 1)
              q >= guard_queue at the envelope : cap = max(cap_min, cap_now - spread_step)
              q >= guard_queue below envelope : cap held (capacity added as nodes)
              always cap <= clamp(1 - cap_gain max(0, B - B_cap), cap_min, 1); expansion never tightens
change gate   Terraform plans only while U >= U_gate (none under direct actuation)
rollback      proposed rollback authorised if S >= S_rollback, or U falling, or security blocks change
replicas      direct actuation: ceil(replicas load / rho*), up unless reducing capacity, down only when needed
routing       traffic shift while network stress > route_network; alerting: one de-duplicated alert;

```

Constant	Value	Constant	Value
rho0	0.914	rho_min	0.627
kI	0.252	kE	0.297
kq	0.795	up_max	2
down_max	1	guard_queue	0.05
power_ceiling	0.95	power_ceiling_backlog	1
spread_step	0.05	backlog_release	0
size_at_full_cap	0	push_release	0.005
push_add	-9	U_gate	0.481
margin	0.006	cap_min	0.706
B_cap	0.4	cap_gain	0.6
S_rollback	0.55	route_network	0.2
down_band	4	down_dwell	4
down_after_add	4	min_nodes	8
max_nodes	40		

Throughput mode replaces the following constants: `backlog_release = True`, `power_ceiling = 9.0`, `power_ceiling_backlog = 9.0`, `spread_step = 0.0`, `B_cap = 9.0`, `size_at_full_cap = True`, `kI = 0.05`, `kE = 0.4`, `cap_min = 0.65`, `down_band = 1`, `down_dwell = 10`, `down_after_add = 3`, `kq = 0.3`, `U_gate = 0.45`, `push_release = 0.01`.

Engine roles: the equation (2) control law, evaluated on the evolved state, gates capacity release: a machine is released only when the push required to hold the stack at full coherence is at or below `push_release`, that is, when the engine reports the stack converged. `f_U` includes the double-well drift and the rate of change of error from equation (1). `I_U` and `E` set the headroom; `U` gates configuration change and power trimming; `B` limits power when power and heat accumulate; `S` and falling `U` authorise rollbacks. Spread-and-throttle uses the model's property that throttled nodes deliver more work per kW; real processors show the same direction through voltage-frequency scaling but with smaller gains. `min_nodes` is carried for completeness; the lower bound is enforced by the stack configuration. Routing is a reflex rule.

6.3 Omni-Compass over Kubernetes and operating modes

In the layered architecture (arms `omni_k8s_protect` and `omni_k8s_throughput`) Kubernetes' autoscalers remain the execution layer. Each interval the governor sets the HPA target to ρ^* ; the HPA computes replicas with its documented rule; Cluster Autoscaler scale-up is retained; the node target is the larger of the Cluster Autoscaler and governor targets when either adds capacity; scale-down, power caps, rollback authority and routing belong to the governor; the combined action set passes through the shield. Operating modes: **power-protect** uses the allocation law above and shield invariant I4; **throughput** releases the power cap to 1 on backlog, disables the power envelope, spread-and-throttle and bath cap, does not apply I4, and sizes capacity at full power (load x `cap_now`) so that power trimming does not trigger machine starts; its power rules match the reference. Throughput-mode constants (`kI`, `kE`, `cap_min`, `down_band`, `down_dwell`, `down_after_add`, `kq`, `U_gate`; values in `results/PREREGISTRATION.json`) were selected by a constrained search on development seeds 1000 and 2000: 110 trials, objective minimum energy, subject to backlog violation $\leq 2.5\%$, time healthy $\geq 88\%$, recovery ≤ 25 minutes, scale reversals ≤ 2.2 and zero non-power invariant violations. Every trial is recorded in `results/development/throughput_search_round*.jsonl`. The lowest-energy trials that failed a constraint did so on reversals (6 to 14 per run); the feasible optimum plateaued near 124 kWh on the development set.

6.4 Safety shield

Every governor action set passes through `omnicompass/shield.py` before execution. Invariants: I1 no capacity expansion during an observed security block; I2 node targets within configured bounds and power cap within [0.65, 1]; I3 no action set both expands capacity and tightens the power cap; I4 no node addition whose projected power stress (observed stress x $(n + k)/n$) exceeds 1; I5 at most 4 nodes changed per interval. Violating actions are clipped or removed and counted. The same invariants are evaluated on the executed actions of every arm, using the true current state, so all arms are scored identically. Scope: I1 governs changes to infrastructure and deployed software (nodes, Terraform plans, rollouts); replica scaling of already-deployed workloads is not blocked by I1. I3 counts replica increases as capacity expansion because they raise power draw.

6.5 Authority

Mode	Engine computes	Stack executes
OBSERVE	every directive, logged	the native managers' own actions
AUTOPILOT	every directive	the governor's shielded actions only
KILLED	every directive, logged	the native managers' own actions from the kill interval onward

7. Benchmark Design

7.1 The stack model

A synthetic discrete-time model (72 intervals of 5 minutes, 20 initial nodes, 8 to 40 nodes, 33 kW site limit, PUE 1.18, Terraform apply delay 8 intervals, node change at most 2 per interval) of the proposal behaviour of Kubernetes, Terraform, HPA/VPA, Karpenter, Argo CD/Rollouts, Istio/Cilium, power/thermal management, alerting and paging. Scenario families: normal variation, demand surge, node failure, thermal stress, network degradation, rollout pressure, security block, telemetry delay and compound events. Extracted byte-for-byte from the reference engine.

7.2 Baselines

native: the reference engine's fragmented-manager model; every manager executes its own proposal. **k8s_ref_50 to k8s_ref_80:** a documented-behaviour reference model of Kubernetes autoscaling with current (non-delayed) metrics. It is not the upstream controllers. HPA desired = $\text{ceil}(\text{current} \times \text{load} / \text{target})$ with target 0.5, 0.6, 0.7 or 0.8 (the target is workload configuration, so all four are run), no change within 10% tolerance, scale-down using the highest recommendation over the 300 s stabilization window; Cluster Autoscaler scale-up sized to unschedulable work, node removal after 10 minutes below 50% utilization and not within 10 minutes of a scale-up; paging after 15 minutes of queue ratio above 0.35; Terraform not used as a live controller; power, rollout, rollback and routing proposals executed as proposed. Not represented: Cluster Autoscaler scheduling simulation, Karpenter consolidation, VPA, scheduling constraints, disruption budgets. The primary tables use target 0.7; Section 8.3 reports all four.

7.3 Omni-Compass arms

Arm	Sensing	Actuation	Shield	Definition
omni_k8s_protect	direct	governor over Kubernetes	on	flagship: layered, power-protect mode
omni_k8s_throughput	direct	governor over Kubernetes	on (no I4)	flagship: layered, throughput mode
omni_k8s_observe	direct	none (Kubernetes acts)	n/a	governor observing Kubernetes; must equal k8s_ref_70
omni_k8s_kill	direct	governor over Kubernetes	on	power-protect, killed at event onset + 2; Kubernetes continues
omni_observe	pipeline	none	n/a	governor in OBSERVE for the whole run
omni_pipeline	pipeline	via Terraform plans	on	AUTOPILOT with delayed telemetry
omni_direct	direct	direct	on	governor replacing Kubernetes' autoscaling decisions
omni_direct_no_dynamics	direct	direct	on	flagship with equations (1) to (7) not evolved
omni_direct_no_shield	direct	direct	off	flagship without the shield
omni_kill	direct	direct	on	flagship killed two intervals after event onset
omni_switch_on	direct	direct	on	OBSERVE until event onset, then AUTOPILOT

7.4 Scoring

All arms are scored by the same functions from stack physics and executed actions. Physical SLA violation: queue ratio > 0.35, power stress > 1.05 or thermal > 1.03. Healthy interval: queue ratio < 0.28, power stress ≤ 1.02, thermal < 0.96 and no contradiction. Recovery: event end to the fourth consecutive healthy interval (360 minutes if not reached). Contradictions: opposing node or replica directions, rollout with rollback, expansion with power tightening, expansion during a security denial, duplicate alerts. Wear: machine start/stop count (sum of absolute node-count changes), power-cap travel (sum of absolute cap changes), thermal travel (sum of absolute thermal changes), scale direction reversals (changes of node or replica direction), machines started, machines stopped, and

machine round trips = (started + stopped - |final - initial node count|) / 2, the number of machines started and later stopped or stopped and later restarted. One-way adjustments, such as shutting down surplus machines, are not round trips. Tail measures: 5th percentile of time healthy, 95th percentiles of recovery, physical SLA violation and energy.

7.5 Protocol

Base constants and engine stack parameters were selected by random search (81 configurations) and local refinement (61) on development seeds 1000 and 2000, minimising energy subject to a service floor. The equation (2) release gate thresholds (0.005 power-protect, 0.01 throughput) and the throughput constants were then selected by a joint search (121 configurations) requiring scale reversals at or below the Kubernetes reference, time healthy ≥ 0.90 , recovery ≤ 17.5 min, backlog $\leq 2\%$, physical violations $\leq 8.5\%$, work completed $\geq 99.9\%$ and zero invariant violations excluding I4; one configuration met every constraint and is used here. A headroom term driven by the same push was tested and rejected (higher energy, no service gain).

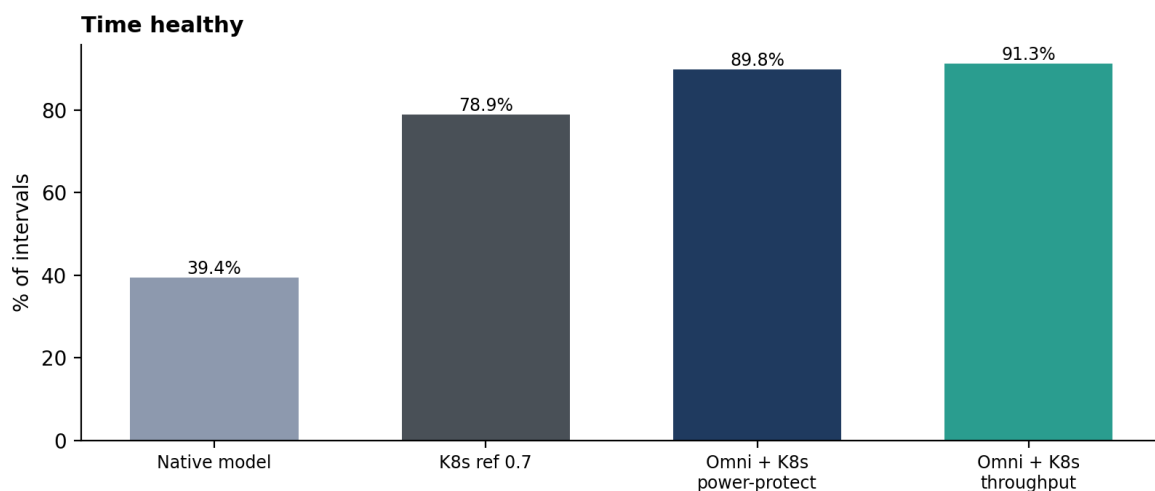
Provenance: results/development/throughput_search_round1.jsonl and throughput_search_round2.jsonl (110 configurations) are result records of an earlier throughput-mode search whose generating script is not in the package. The throughput constants were subsequently re-evaluated by the recorded joint search (results/development/joint_search.py, JOINT_121.jsonl), in which no configuration met all constraints with lower energy.

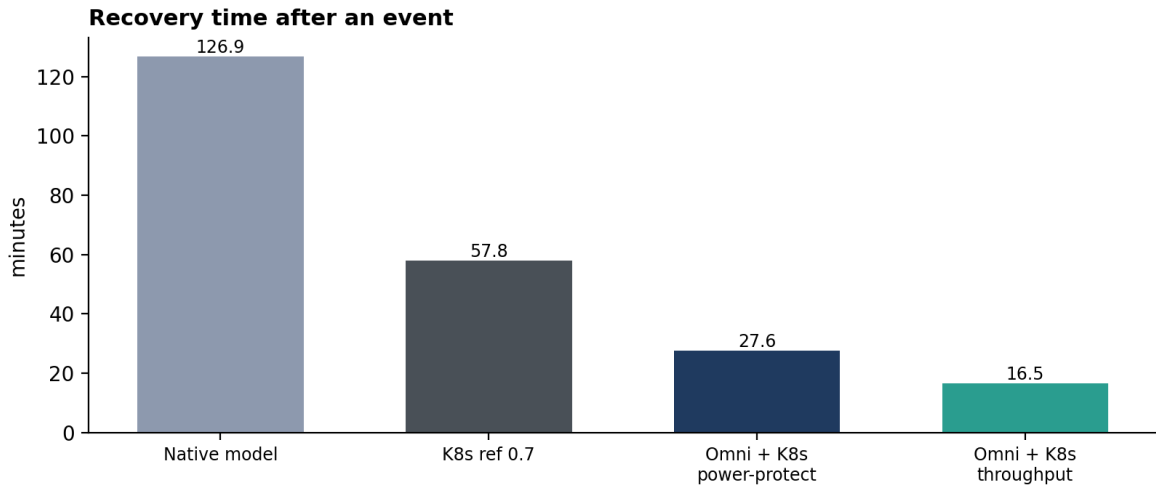
Added after the held-out runs, without changing any pre-registered file: benchmarks/multiplicity.py (primary endpoint energy_kwh; secondary metrics Holm-adjusted), a two-mode soak with memory checkpoints, a C++ contract smoke test, and verify.py --full-replay.

Development seeds: 1000, 2000, 900000001, 224297269, 314159265, 271828182, 161803398, 141421356, 173205080, 223606797, 236067977, 244948974, 264575131, 282842712, 316227766, 331662479. The allocation law, shield, baselines and benchmark code were frozen and their SHA-256 and program fingerprints recorded in results/PREREGISTRATION.json before the held-out seeds 346410161 and 360555127 (500 scenarios each) were run. Paired 95% intervals are bootstrap intervals over scenarios (4,000 resamples). A result is called better or worse only when the interval excludes zero.

8. Results

Figures show means over both held-out seeds; tables report each seed separately with paired significance. Verdicts compare each Omni-Compass mode with the Kubernetes reference at HPA target 0.7.





8.1 Held-out seed 346410161 (500 scenarios)

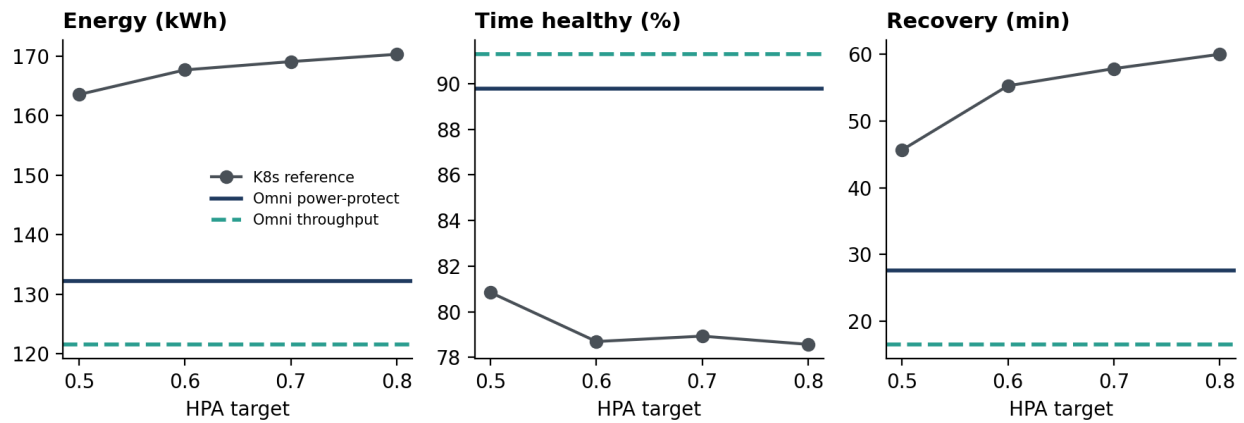
Measure	K8s ref 0.7	power-protect	throughput	protect vs ref	throughput vs ref
Time healthy	79.3%	89.9%	91.2%	better	better
Recovery (minutes)	57.2	27.9	17.8	better	better
Scenarios recovered	91.8%	96.8%	99.2%	better	better
Physical SLA violation	13.9%	8.8%	7.4%	better	better
backlog	2.5%	6.7%	1.6%	worse	better
power	12.8%	3.5%	6.9%	better	better
heat	8.0%	0.3%	2.7%	better	better
Work completed	99.82%	99.63%	99.98%	worse	better
Energy (kWh)	168.9	132.2	121.7	better	better
Node-hours	135.5	160.1	140.2	worse	worse
Contradictions per run	9.6	0.0	0.0	better	better
Invariant violations I1-I5	9.56	0.00	2.72	better	better
Invariant violations excl. I4	6.91	0.00	0.00	better	better
Expansions during security block	0.92	0.00	0.00	better	better
Scale direction reversals	2.19	3.06	1.93	worse	better
Machines started	4.8	10.7	6.5	count	count
Machines stopped	1.2	2.6	3.3	count	count
Machine round trips	1.1	2.6	1.7	worse	worse
Power-cap travel	0.24	0.31	0.88	worse	worse
Thermal travel	0.75	0.59	0.63	better	better

8.2 Held-out seed 360555127 (500 scenarios)

Measure	K8s ref 0.7	power-protect	throughput	protect vs ref	throughput vs ref
Time healthy	78.5%	89.7%	91.4%	better	better
Recovery (minutes)	58.4	27.2	15.2	better	better

Measure	K8s ref 0.7	power-protect	throughput	protect vs ref	throughput vs ref
Scenarios recovered	91.8%	97.2%	100.0%	better	better
Physical SLA violation	14.9%	9.0%	7.1%	better	better
backlog	2.6%	7.0%	1.5%	worse	better
power	13.8%	3.7%	6.5%	better	better
heat	8.8%	0.4%	2.5%	better	better
Work completed	99.97%	99.83%	100.00%	worse	better
Energy (kWh)	169.2	132.3	121.5	better	better
Node-hours	136.1	159.3	140.8	worse	worse
Contradictions per run	9.4	0.0	0.0	better	better
Invariant violations I1-I5	9.44	0.00	2.69	better	better
Invariant violations excl. I4	6.80	0.00	0.00	better	better
Expansions during security block	0.86	0.00	0.00	better	better
Scale direction reversals	2.22	3.08	1.92	worse	better
Machines started	4.8	10.6	6.6	count	count
Machines stopped	1.2	2.6	3.4	count	count
Machine round trips	1.1	2.6	1.8	worse	worse
Power-cap travel	0.23	0.30	0.89	worse	worse
Thermal travel	0.75	0.59	0.63	better	better

8.3 Sensitivity to the reference HPA target



Measure	power-protect better in	throughput better in	throughput worse in
Time healthy	8/8	8/8	0/8
Recovery	8/8	8/8	0/8
Physical SLA	8/8	8/8	0/8
Backlog violation	0/8	8/8	0/8
Work completed	0/8	6/8	1/8
Energy	8/8	8/8	0/8
Reversals	0/8	4/8	4/8

Measure	power-protect better in	throughput better in	throughput worse in
Round trips	2/8	2/8	6/8
Thermal travel	8/8	8/8	0/8

Counts are over 2 seeds x 4 HPA targets. Human pages are not reported as a result: the governor issues no page actions by design.

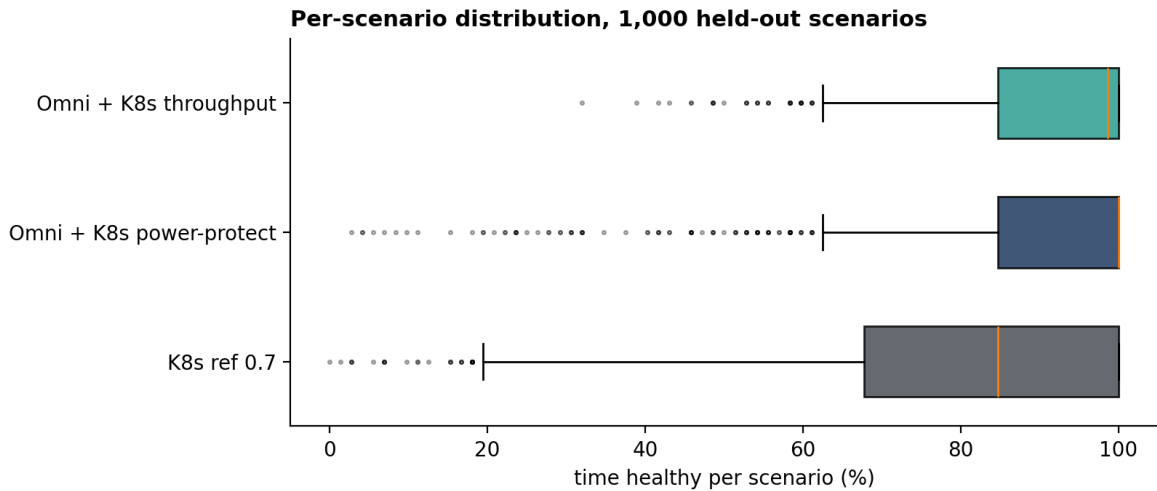
8.4 Primary endpoint and multiplicity

Primary comparison: throughput mode vs the Kubernetes reference at HPA target 0.7. Primary metric: energy per run. Secondary metrics are tested with a paired sign-flip permutation test (10,000 permutations) and Holm-adjusted across all secondary metrics and both seeds (24 tests). The smallest attainable p-value is 1/10,001.

Seed	Energy difference (kWh)	p	Scenarios better
346410161	-47.20	1.0e-04	493/500
360555127	-47.69	1.0e-04	498/500

Seed	Metric	Difference	Holm p	Verdict
346410161	time_healthy	+0.1185	2.4e-03	better
346410161	recovery_minutes	-39.4000	2.4e-03	better
346410161	recovered	+0.0740	2.4e-03	better
346410161	sla_violation_physical	-0.0650	2.4e-03	better
346410161	violation_backlog	-0.0098	2.4e-03	better
346410161	violation_power	-0.0593	2.4e-03	better
346410161	violation_heat	-0.0533	2.4e-03	better
346410161	availability	+0.0016	7.6e-03	better
346410161	invariant_violations_ex_power	-6.9060	2.4e-03	better
346410161	scale_reversals	-0.2560	2.4e-03	better
346410161	machine_round_trips	+0.5960	2.4e-03	worse
346410161	thermal_travel	-0.1206	2.4e-03	better
360555127	time_healthy	+0.1286	2.4e-03	better
360555127	recovery_minutes	-43.1900	2.4e-03	better
360555127	recovered	+0.0820	2.4e-03	better
360555127	sla_violation_physical	-0.0771	2.4e-03	better
360555127	violation_backlog	-0.0117	2.4e-03	better
360555127	violation_power	-0.0725	2.4e-03	better
360555127	violation_heat	-0.0625	2.4e-03	better
360555127	availability	+0.0003	1.3e-01	not significant
360555127	invariant_violations_ex_power	-6.8040	2.4e-03	better
360555127	scale_reversals	-0.2980	2.4e-03	better
360555127	machine_round_trips	+0.7210	2.4e-03	worse
360555127	thermal_travel	-0.1226	2.4e-03	better

8.5 Worst cases



Arm	Healthy p05	Healthy min	Recovery p95 (min)	Physical SLA p95	Energy p95 (kWh)
native	1.4%	0.0%	360	51.4%	192.7
k8s_ref_70	29.1%	1.4%	360	52.8%	226.5
omni_k8s_protect	48.6%	2.8%	105	45.8%	169.6
omni_k8s_throughput	66.5%	31.9%	50	30.6%	187.0

Seed 346410161.

8.6 Attribution

Arm	Healthy	Phys. SLA	Backlog	Recovery	Energy	Round trips	Inv. viol.
native	39.5%	17.9%	13.3%	127.4	131.9	34.6	28.35
k8s_ref_70	79.3%	13.9%	2.5%	57.2	168.9	1.1	9.56
omni_observe	39.5%	17.9%	13.3%	127.4	131.9	34.6	28.35
omni_k8s_observe	79.3%	13.9%	2.5%	57.2	168.9	1.1	9.56
omni_k8s_protect	89.9%	8.8%	6.7%	27.9	132.2	2.6	0.00
omni_k8s_throughput	91.2%	7.4%	1.6%	17.8	121.7	1.7	2.72
omni_k8s_kill	82.4%	12.4%	2.2%	54.2	146.2	1.1	4.02
omni_direct	90.2%	8.8%	7.9%	28.6	129.5	2.7	0.00
omni_direct_no_dynamics	90.3%	8.6%	7.4%	27.5	131.5	0.0	0.00
omni_direct_no_shield	90.1%	8.2%	6.6%	27.0	130.3	2.7	0.67
omni_pipeline	89.0%	9.7%	8.0%	28.7	133.6	3.4	1.27
omni_kill	68.0%	9.4%	2.1%	90.2	124.6	14.9	11.28
omni_switch_on	75.8%	12.5%	12.1%	34.1	127.4	12.7	1.91

Seed 346410161. omni_observe is bit-identical to native in every held-out scenario; omni_k8s_observe (governor observing Kubernetes) is bit-identical to Kubernetes alone in 500/500 and 500/500. omni_k8s_kill runs power-protect mode until two intervals after event onset, after which Kubernetes alone continues. omni_k8s_protect vs omni_direct measures the value of keeping Kubernetes' autoscalers as the execution layer; omni_direct vs

omni_pipeline measures direct sensing and actuation; omni_direct vs omni_direct_no_dynamics measures the contribution of evolving equations (1) to (7): energy 129.5 vs 131.5 kWh, round trips 2.7 vs 0.0, healthy 90.2% vs 90.3%. The engine dynamics contribute a modest share of the gain; direct wiring, single authority, the allocation law and the shield carry most of it. The kill and switch-on arms omni_kill and omni_switch_on are built on omni_direct, where the native managers resume after a kill.

Engine contribution in the flagship (throughput mode, both seeds). With equations (1) to (7) evolved and the equation (2) release gate: energy 121.6 kWh, reversals 1.93, round trips 1.7, healthy 91.3%. Gate removed: energy 120.0, reversals 3.18, round trips 3.3, healthy 91.0%. Evolution removed: energy 125.3, reversals 0.00, round trips 0.0, healthy 90.4%. Paired comparisons are in the summary files under omni_k8s_throughput_vs_omni_k8s_throughput_no_gate and omni_k8s_throughput_vs_omni_k8s_throughput_no_dynamics.

8.7 Fleet component model

Role	Share of fleet vCPU	Share of fleet energy
execution	1.16%	0.22 to 1.01%
observability	0.42%	0.22 to 0.51%
security	9.38%	2.33 to 8.67%
decision	0.02%	0.01 to 0.02%
idle capacity	92%	18%

100,000 nodes in 20 clusters, 64 cores and 256 GiB per node, 30 pods per node, 8% mean CPU utilization, sidecar service mesh. Energy per resource: Cloud Carbon Footprint coefficients (SPECpower-derived), AMD EPYC 3rd generation: 0.434 W min, 1.948 W max per vCPU; memory 0.392 W per GB (arXiv 2510.26413). Decision components consume a negligible share of the fleet; their cost is the capacity they leave idle or padded, which is the quantity the governor changes. The service mesh is the largest single component and is a security function retained under the keep-or-remove rule.

8.8 Fleet harness at 15-second resolution

fleet/ is a second, independent harness in which the Kubernetes execution layer runs at its documented cadence: metrics-server (15 s lag, usage over request), HPA (ratio law, 10% tolerance, 300 s scale-down window, scale-up limited to max(4 pods, 100%) per 15 s), Cluster Autoscaler (pending scale-up; removal after 10 min below 50% request utilization, not within 10 min of a scale-up) and Karpenter-lite (exact provisioning for pending requests, consolidation every 30 s). The plant has many workloads per cluster with CPU requests and actual usage, request packing, identical node boot delay for every arm, power, heat and a site power limit. Vessels: web services (12 HPA workloads, diurnal load with bursts), batch/HPC job queue, GPU training (8-GPU nodes), and a multi-cluster fleet (4 clusters sharing one site power budget). In omni_fleet the governor is the node-pool and power authority in place of the Cluster Autoscaler, with the HPA retained as the replica controller; this is a different question from governing on top of an unchanged HPA and Cluster Autoscaler (omni_target). The single-authority arms are the comparison reported below, deciding every 60 s: HPA runs with target ρ^* , the Cluster Autoscaler is not run, nodes never fall below what current pod requests need, and actions pass through the shield. All workload series are synthetic.

Fleet-mode constants were selected by random search (70 + 90 configurations) on development scenarios (seeds 101-104 of every vessel). Selection rule, fixed before selection: lowest mean energy relative to HPA 0.7 + Cluster Autoscaler among configurations that beat HPA 0.7 + Karpenter-lite on both energy and node reversals in every vessel, with time healthy ≥ 0.99 (web, multi, batch) and work completed within 0.2% of the baseline. Eight configurations met the rule. The selected configuration is record 55 of SEARCH_70.jsonl (script search_70.py).

search_90.py explored an additional up_band parameter; no selected configuration uses it and it has been removed from the governor, so search_90.py documents the exploration but no longer runs against the current code.

Vessel	Setting and baseline	Energy (kWh)	Node reversals	Time healthy	Work completed
web	energy-first vs HPA 0.7 + CA	13.6 to 8.9 (-34.5%) better	1.1 to 4.3 worse	100.0% to 100.0% not significant	100.00% to 99.99%
web	energy-first vs HPA 0.7 + Karpenter-lite	10.8 to 8.9 (-16.9%) better	6.8 to 4.3 better	100.0% to 100.0% not significant	100.00% to 99.99%
web	balanced vs HPA 0.7 + CA	13.6 to 9.1 (-33.2%) better	1.1 to 4.0 worse	100.0% to 100.0% not significant	100.00% to 99.99%
web	balanced vs HPA 0.7 + Karpenter-lite	10.8 to 9.1 (-15.3%) better	6.8 to 4.0 better	100.0% to 100.0% not significant	100.00% to 99.99%
web	wear-first vs HPA 0.7 + CA	13.6 to 9.6 (-29.7%) better	1.1 to 3.6 worse	100.0% to 100.0% not significant	100.00% to 99.99%
web	wear-first vs HPA 0.7 + Karpenter-lite	10.8 to 9.6 (-10.9%) better	6.8 to 3.6 better	100.0% to 100.0% not significant	100.00% to 99.99%
web	park vs HPA 0.7 + CA	13.6 to 11.5 (-15.9%) better	1.1 to 0.0 better	100.0% to 100.0% not significant	100.00% to 99.99%
web	park vs HPA 0.7 + Karpenter-lite	10.8 to 11.5 (+6.7%) worse	6.8 to 0.0 better	100.0% to 100.0% not significant	100.00% to 99.99%
multi	energy-first vs HPA 0.7 + CA	54.1 to 35.6 (-34.3%) better	5.5 to 18.1 worse	100.0% to 99.8% worse	100.00% to 99.99%
multi	energy-first vs HPA 0.7 + Karpenter-lite	42.7 to 35.6 (-16.6%) better	25.4 to 18.1 better	100.0% to 99.8% worse	100.00% to 99.99%
multi	balanced vs HPA 0.7 + CA	54.1 to 36.3 (-33.0%) better	5.5 to 15.5 worse	100.0% to 99.8% worse	100.00% to 99.99%
multi	balanced vs HPA 0.7 + Karpenter-lite	42.7 to 36.3 (-15.0%) better	25.4 to 15.5 better	100.0% to 99.8% worse	100.00% to 99.99%
multi	wear-first vs HPA 0.7 + CA	54.1 to 38.1 (-29.6%) better	5.5 to 14.5 worse	100.0% to 99.8% worse	100.00% to 99.99%
multi	wear-first vs HPA 0.7 + Karpenter-lite	42.7 to 38.1 (-10.6%) better	25.4 to 14.5 better	100.0% to 99.8% worse	100.00% to 99.99%
multi	park vs HPA 0.7 + CA	54.1 to 45.7 (-15.6%) better	5.5 to 0.0 better	100.0% to 99.9% worse	100.00% to 99.99%
multi	park vs HPA 0.7 + Karpenter-lite	42.7 to 45.7 (+7.0%) worse	25.4 to 0.0 better	100.0% to 99.9% worse	100.00% to 99.99%
batch	energy-first vs HPA 0.7 + CA	66.8 to 55.3 (-17.2%) better	1.4 to 13.6 worse	100.0% to 100.0% not significant	100.00% to 99.98%
batch	energy-first vs HPA 0.7 + Karpenter-lite	61.3 to 55.3 (-9.7%) better	24.9 to 13.6 better	100.0% to 100.0% better	100.00% to 99.98%
batch	balanced vs HPA 0.7 + CA	66.8 to 58.5 (-12.4%) better	1.4 to 5.7 worse	100.0% to 100.0% not significant	100.00% to 99.98%
batch	balanced vs HPA 0.7 + Karpenter-lite	61.3 to 58.5 (-4.5%) better	24.9 to 5.7 better	100.0% to 100.0% better	100.00% to 99.98%
batch	wear-first vs HPA 0.7 + CA	66.8 to 58.6 (-12.3%) better	1.4 to 4.9 worse	100.0% to 100.0% not significant	100.00% to 99.98%
batch	wear-first vs HPA 0.7 + Karpenter-lite	61.3 to 58.6 (-4.3%) better	24.9 to 4.9 better	100.0% to 100.0% better	100.00% to 99.98%
batch	park vs HPA 0.7 + CA	66.8 to 56.6 (-15.3%) better	1.4 to 0.0 better	100.0% to 100.0% not significant	100.00% to 99.98%

Vessel	Setting and baseline	Energy (kWh)	Node reversals	Time healthy	Work completed
batch	park vs HPA 0.7 + Carpenter-lite	61.3 to 56.6 (-7.7%) better	24.9 to 0.0 better	100.0% to 100.0% better	100.00% to 99.98%
gpu	energy-first vs HPA 0.7 + CA	193.7 to 184.2 (-4.9%) better	2.8 to 5.1 worse	43.3% to 46.4% better	90.16% to 90.15%
gpu	energy-first vs HPA 0.7 + Carpenter-lite	189.3 to 184.2 (-2.7%) better	11.2 to 5.1 better	44.1% to 46.4% better	90.14% to 90.15%
gpu	balanced vs HPA 0.7 + CA	193.7 to 184.0 (-5.0%) better	2.8 to 4.2 worse	43.3% to 47.0% better	90.16% to 90.14%
gpu	balanced vs HPA 0.7 + Carpenter-lite	189.3 to 184.0 (-2.8%) better	11.2 to 4.2 better	44.1% to 47.0% better	90.14% to 90.14%
gpu	wear-first vs HPA 0.7 + CA	193.7 to 184.9 (-4.5%) better	2.8 to 3.0 not significant	43.3% to 46.7% better	90.16% to 90.15%
gpu	wear-first vs HPA 0.7 + Carpenter-lite	189.3 to 184.9 (-2.3%) better	11.2 to 3.0 better	44.1% to 46.7% better	90.14% to 90.15%
gpu	park vs HPA 0.7 + CA	193.7 to 184.3 (-4.9%) better	2.8 to 0.0 better	43.3% to 47.0% better	90.16% to 90.15%
gpu	park vs HPA 0.7 + Carpenter-lite	189.3 to 184.3 (-2.7%) better	11.2 to 0.0 better	44.1% to 47.0% better	90.14% to 90.15%
gpu_always_on	energy-first vs HPA 0.7 + CA	193.3 to 184.3 (-4.7%) better	0.0 to 0.0 not significant	43.6% to 47.0% better	90.15% to 90.15%
gpu_always_on	energy-first vs HPA 0.7 + Carpenter-lite	190.4 to 184.3 (-3.2%) better	0.0 to 0.0 not significant	44.8% to 47.0% better	90.15% to 90.15%
gpu_always_on	balanced vs HPA 0.7 + CA	193.3 to 184.2 (-4.7%) better	0.0 to 0.0 not significant	43.6% to 47.4% better	90.15% to 90.14%
gpu_always_on	balanced vs HPA 0.7 + Carpenter-lite	190.4 to 184.2 (-3.2%) better	0.0 to 0.0 not significant	44.8% to 47.4% better	90.15% to 90.14%
gpu_always_on	wear-first vs HPA 0.7 + CA	193.3 to 185.1 (-4.2%) better	0.0 to 0.0 not significant	43.6% to 47.1% better	90.15% to 90.15%
gpu_always_on	wear-first vs HPA 0.7 + Carpenter-lite	190.4 to 185.1 (-2.8%) better	0.0 to 0.0 not significant	44.8% to 47.1% better	90.15% to 90.15%
gpu_always_on	park vs HPA 0.7 + CA	193.3 to 184.3 (-4.7%) better	0.0 to 0.0 not significant	43.6% to 47.0% better	90.15% to 90.15%
gpu_always_on	park vs HPA 0.7 + Carpenter-lite	190.4 to 184.3 (-3.2%) better	0.0 to 0.0 not significant	44.8% to 47.0% better	90.15% to 90.15%

Held-out: 30 scenarios per vessel from seed 700000; paired bootstrap 95% intervals; a verdict is stated only when the interval excludes zero. Observe mode is bit-identical to HPA 0.7 + Cluster Autoscaler in every scenario of every vessel. Differences in time healthy and work completed of 0.1% or less can be statistically significant and are reported as measured.

Vessel	Ablation	Full engine energy minus ablation	Time healthy
web	equations (1)-(7) not evolved	-0.02 kWh not significant	-0.0 pts not significant
web	equation (2) release gate removed	+0.00 kWh not significant	+0.0 pts not significant
multi	equations (1)-(7) not evolved	+0.01 kWh not significant	-0.1 pts not significant
multi	equation (2) release gate removed	+0.00 kWh not significant	+0.0 pts not significant
batch	equations (1)-(7) not evolved	-8.91 kWh better	+0.0 pts not significant
batch	equation (2) release gate removed	+0.00 kWh not significant	+0.0 pts not significant

Vessel	Ablation	Full engine energy minus ablation	Time healthy
gpu	equations (1)-(7) not evolved	-9.41 kWh better	+4.0 pts better
gpu	equation (2) release gate removed	+0.00 kWh not significant	+0.0 pts not significant
gpu_always_on	equations (1)-(7) not evolved	-9.62 kWh better	+4.1 pts better
gpu_always_on	equation (2) release gate removed	+0.00 kWh not significant	+0.0 pts not significant

Families: web, multi, batch and gpu are elastic (nodes may be powered off); gpu_always_on is always-on (powering a node off is not permitted, so every arm may only park idle nodes: Ready, 25% of idle power, wake within one tick). The park setting executes the energy-first law's capacity reductions as parking instead of power-off. Machines stopped counts power-offs only; park moves are counted separately.

Settings: energy-first uses the lenient release threshold selected for energy (the equation (2) gate does not bind); balanced and wear-first activate the equation (2) gate in both directions (a machine is released only when the push reports convergence and, without backlog, added only when the push reports need), trading energy for fewer node reversals. The engine's evolution reduces energy in the batch and GPU vessels and has no significant effect in the web and multi-cluster vessels (ablations above, energy-first).

8.9 Recorded workloads (PlanetLab)

The fleet harness was run with the frozen laws, without retuning, on 9 recorded PlanetLab/CoMon VM CPU traces (30 scenarios from seed 800000; each scenario assigns traces, requests and peak pod counts to 12 workloads). These are recorded utilization shapes with a declared scale: the mapping from CPU percent to cores (percent / 100 x peak pods x request, 12 workloads) is declared in fleet/planetlab.py, so the load level is a modelling choice, not a measurement. Copies supplied with an uploaded harness package (omnicompass_k8s_harness.zip); the repository could not be re-downloaded in the build environment, so byte identity with the upstream files is not independently confirmed. Observe mode was bit-identical to HPA 0.7 + CA in 30/30 scenarios.

Arm	Energy (kWh)	vs HPA + CA	vs Karpenter-lite	Time healthy	Node reversals
k8s_hpa70_ca	7.51	+0.0%	+32.0%	100.00%	1.10
k8s_hpa70_karpenter	5.69	-24.2%	+0.0%	100.00%	1.93
omni_target	7.46	-0.7% better	+31.1% worse	99.45%	0.80
omni_fleet	4.46	-40.6% better	-21.7% better	99.92%	2.03
omni_fleet_balanced	4.69	-37.5% better	-17.5% better	99.92%	1.87
omni_fleet_wear	5.18	-31.0% better	-8.9% better	99.92%	1.57
omni_fleet_park	7.60	+1.3% worse	+33.6% worse	99.92%	0.00
omni_fleet_no_dynamics	4.47	-40.5% better	-21.4% better	99.95%	2.03

On recorded workloads the governor's energy reduction relative to both baselines is significant in the energy-first, balanced and wear-first settings; time healthy (by 0.08 points) and work completed (by 0.01%) are significantly lower; node reversals are not significantly different from Karpenter-lite. Parking does not reduce energy at these low utilizations. Evolving equations (1)-(7) has no significant effect on this web-type workload.

8.10 HPA agreement across implementations

Three implementations of the HPA replica law are compared on every step of recorded streams: the fleet harness (a 20-sample recommendation ring), an independent C++ implementation, and an externally written Python

implementation that keeps recommendations by timestamp. Upstream Kubernetes keeps recommendations whose timestamp is after the cutoff (exclusive); at a fixed 15 s cadence the 20-sample ring is exactly that window, (now - 300 s, now]. The external implementation, with an inclusive boundary, differs on 0.39% of steps; with the exclusive boundary all three agree on every step (tests/test_hpa_three_way.py). The ring would diverge from a timestamp window only if sampling were irregular.

8.11 Savings projection (method, not measurement)

benchmarks/savings.py and cpp/tools/savings.cpp apply the energy reduction of the energy-first governor relative to HPA 0.7 + Karpenter-lite (mean and paired 95% interval) to declared fleet profiles: annual energy = nodes x average kW x PUE x 8,760 h; cost and CO2 follow from the declared price and carbon intensity. GPU vessels use a declared 6 kW per node. The reductions are simulation results; the profiles are examples; the output is a projection, not a measurement.

Profile	Vessel	Saved MWh / yr (mid)	Saved USD / yr (low to high)	Saved t CO2 / yr (mid)
cluster_1k_nodes	web	830	85,119 to 115,022	332
cluster_1k_nodes	multi	816	90,111 to 105,643	326
cluster_1k_nodes	batch	478	50,461 to 64,059	191
cluster_1k_nodes	gpu	1,997	186,464 to 293,466	799
cluster_1k_nodes	gpu_always_on	2,366	219,320 to 353,406	947
cluster_1k_nodes	planetlab_shapes	1,062	121,078 to 132,880	425
fleet_100k_nodes	web	77,066	5,269,265 to 7,120,400	30,827
fleet_100k_nodes	multi	75,745	5,578,295 to 6,539,801	30,298
fleet_100k_nodes	batch	44,396	3,123,800 to 3,965,550	17,758
fleet_100k_nodes	gpu	185,419	11,542,985 to 18,166,925	74,168
fleet_100k_nodes	gpu_always_on	219,743	13,576,936 to 21,877,542	87,897
fleet_100k_nodes	planetlab_shapes	98,635	7,495,331 to 8,225,924	39,454

8.12 Live controller

omni_controller/controller.py runs the fleet-mode governor against a real cluster through kubectl, every 60 s by default, in three modes: observe (decisions logged, no writes; the default), target (each HPA's CPU averageUtilization set to ρ^* , bounded to 50-95% and changed only by at least 3 points, original recorded in an annotation; node management unchanged) and nodepool (one node pool sized to the governor's recommendation through a configured command, never below what running pod requests or current usage need, step-limited by the shield; the Cluster Autoscaler must not manage that pool). --dry-run logs writes without executing them. The kill switch (a file or OMNI_KILL=1) restores every changed HPA target from its annotation, including after a restart, and returns to observe. Every decision and write is appended to an audit log. deploy/ contains a Dockerfile and manifests with separate read-only (observe) and HPA-write (target) permissions. The controller is tested against a fake kubectl (tests/test_omni_controller.py); it has not been run against a real cluster. The scheduling floor counts running and pending pods' CPU requests plus a headroom margin (default 50%); between governor decisions a 15-second check adds the nodes that pending pods need in one step.

8.12a End-to-end self-pilot

pilot/selfpilot.py runs the shipped controller in nodepool mode against a simulated cluster (the fleet harness web workload, request packing, 90 s node boot, power) that answers its kubectl reads and applies its HPA patches and

node-pool resizes, for one day, and scores it against HPA 0.7 + Cluster Autoscaler on identical traffic with pilot/score.py. Headroom trade-off measured on seed 424242 (change vs baseline; pending-pod minutes per hour, baseline 4.0): headroom 0: energy per core-hour -17%, pending 24.6; 0.10: -17%, 20.6; 0.25: -15%, 13.4; 0.50 (default): -7.7%, 5.0 (no significant difference), node-hours per core-hour -14%. HPA shortfall minutes are higher than baseline at every headroom setting (about +1.2 minutes per hour); the cause is open. Running this self-pilot found and fixed two controller faults: the scheduling floor ignored pending pods, and capacity for pending pods waited for the next governor decision.

8.13 Scoring a pilot

pilot/score.py compares a baseline capture with an Omni-Compass capture from the same cluster (periods or matched node pools), normalising by work done: node-hours and energy per used CPU core-hour, utilisation, pending-pod minutes per hour and HPA shortfall minutes per hour, each with a bootstrap 95% interval over hourly blocks. Energy is measured when captures include power, otherwise modelled from declared node power. It is tested to detect a real efficiency gain, to report no significant difference between identical clusters and to detect a service regression (tests/test_pilot_score.py).

8.14 Paths for real data

Three components connect the package to real systems; each is tested on generated inputs in the exact real formats (tests/test_fleet_reald_data_paths.py). No real capture or recorded trace has been run inside this package, because the build environment has no cluster and no network access to the trace repository.

Component	Input	Output
fleet/capture/kube_capture.sh	a live cluster, read-only (kubectl get/top, jq); optional site power command	CSV every 15 s: nodes, allocatable, requested and used CPU, pending pods, HPA current and desired replicas, power
fleet/capture_replay.py	a capture CSV	the fleet-mode governor in OBSERVE: recommended node count, power cap and HPA target per decision; counterfactual node-hours and, with a power model or captured power, estimated energy
fleet/planetlab.py	a directory of PlanetLab traces (one integer percent per line)	every fleet arm on recorded workload shapes with the frozen laws

The capture replay carries the recommended fleet forward as the governor's own counterfactual state, never below the node count that running pod requests or current usage require. Its output is a recommendation, not a measurement: the cluster's response to the recommended fleet is not modelled.

9. Reproduction

```
# full verification
python verify.py

# one held-out stack run
python benchmarks/stack_benchmark.py --seed 161803398 --scenarios 500 --out out/

# core evidence, fleet model
python benchmarks/core_evidence.py
python benchmarks/fleet_overhead.py

# soak test (C++)
cmake -S cpp -B build && cmake --build build && build/oc_soak 100000000
```

```
# rebuild manual and evaluation kit
python docs/build_manual.py
python docs/build_kit.py
```

10. Open Obligations

- Production evidence: all stack results are from the synthetic model; replay of published production traces (Google, Alibaba, Azure) and the pilot protocol are required.
- Stack model validity: proposal rules, power and thermal physics, the throttling efficiency relation and delays are not calibrated to measured systems.
- Boundedness theorem: a forward-invariance or input-to-state-stability proof for the sampled-data governor loop. A computer-assisted interval-arithmetic proof was attempted; it did not close, because interval bounds on the cubic U term widen across the ten micro steps. Taylor-model or affine arithmetic is the next method. Boundedness is supported empirically by the soak tests and adversarial runs.
- Barrier certificates: the shield is a runtime monitor; a barrier-certificate proof that the shielded loop keeps the stack in the safe set under the model is open.
- Wear: see Chapter 8 for scale direction reversals and machine round trips in each mode relative to the reference; power-protect mode reverses direction more often than the reference.
- Fleet harness: workloads are synthetic; the execution layer reproduces documented behaviour, not the upstream binaries; scheduling is fluid packing without affinity, disruption budgets or multi-zone placement; GPU throughput scales linearly with the power cap.
- Engine mechanism: the six-state engine contributes through the equation (2) release gate and the state-derived headroom; the governor's capacity sizing, power trimming and single authority remain rule-based.
- Power-protect mode: backlog violations and work completed are worse than the reference, because the site power limit is enforced and the reference does not enforce it.
- Baseline fidelity: the Kubernetes baseline is a documented-behaviour reference model; running the upstream Cluster Autoscaler, Karpenter, HPA and VPA controllers against the same scenarios is open.
- Power-protect constants were set by development tuning, not by constrained search; throughput constants come from a 110-trial random and local search and are not proven optimal.
- Multi-cluster, GPU-cluster and facility-level vessels are not yet built.
- Independent reimplementations from this manual has not been performed.

Appendix A. File Register

File	SHA-256
.github/workflows/verify.yml	8e98889cef80143b82140bd1d35d1bdb...
.gitignore	4751c611f0b51f512efa575ac6d27716...
CONTRIBUTING.md	9334855d0c300e0feb2477c4ab0d27a0...
LICENSE	fe5da67a0c6cbb24d27278a26a82d705...
README.md	f674c45acef63a715c71e930bef3f872...
SECURITY.md	15e687da30ed9d5ce6f00d4b12bfa3b6...
benchmarks/__init__.py	e3b0c44298fc1c149afb4c8996fb924...
benchmarks/core_evidence.py	5013dcc81db1c3eb6b05043675ea8e5b...
benchmarks/fleet_overhead.py	232accd3a645ba881ba71916e13ddb2c...
benchmarks/multiplicity.py	5ef32b53e6f36f97d09d56783c32cc8b...
benchmarks/savings.py	f6bb43b9d6bb2638a001731cfaac4274...
benchmarks/stack_benchmark.py	012cce208ce893f542e55cae70b01a30...
cpp/CMakeLists.txt	39feb5bdf5bfd63fb6d1da330d18a3b47...
cpp/include/omnicompass/core.hpp	a5268199fef7e788940d9fa27278d0d1...
cpp/include/omnicompass/governor.hpp	42a7a157d21de43e83421ac45b842372...
cpp/include/omnicompass/hpa.hpp	9fbffe76746981e2cb32a9b3b43b6d36...
cpp/include/omnicompass/shield.hpp	ea047bb2fca4f4997c31d45f3c18596e...
cpp/scripts/compare_results.py	9f8b7ad299eeb5bdefca70e62796966c...
cpp/src/core.cpp	d7deead6f47d54d5b68e08b98db5e15a...
cpp/src/csv.cpp	841147a0c7a19593885ed38ea2954f96...
cpp/src/governor.cpp	8fcedb561ba6101881429b6309bbb2a8...
cpp/src/hpa.cpp	91e9b487afe6e8db4f3b355516a0ec85...
cpp/src/shield.cpp	f1d0211b54d53e088568591d1528f000...
cpp/tools/run_fixture.cpp	c4931db605bb24c077e5740fa0f0ed63...
cpp/tools/run_governor.cpp	18b4c7ff3a92c52f29ee3533dd3eed82...
cpp/tools/run_hpa.cpp	7c2675ba84dda153a84ba1a1cc986b83...
cpp/tools/run_shield.cpp	a7edc9a4982ed5e45206b622bad7ef65...
cpp/tools/savings.cpp	a1f0f72285d3c35510678ae44e41238e...
cpp/tools/smoke.cpp	69d47e3fa386eb7ea5aaffaef75715c1...
cpp/tools/soak.cpp	b5ce02b4886ee59300b6ed4dc2d78a86...
deploy/Dockerfile	5302a3029da438df23cf2d621f0f9a79...
deploy/controller.yaml	e7b2dcf7ea3afbd1f425c18dcadf5533...
deploy/rbac-observe.yaml	3492f3ee21af082e20d504aea1ab9df8...
deploy/rbac-target.yaml	d50af7b19da203da9eefbbf4c4fe80ff...
docs/CLAIMS_REGISTER.md	7119a63fd5435be624c27b87889576fd...
docs/DOMAIN_MAP.md	d97a10e22a5870d4f572594309bef889...
docs/DUE_DILIGENCE.md	7cc135758580cd01513a5021def08db0...

File	SHA-256
docs/PILOT_PROTOCOL.md	3ff68194a58d93ce4d0e9235501b5ffa...
docs/build_figures.py	b5db404f0f75823a11eab964e80866b9...
docs/build_kit.py	6610ad5a689d45c766271e3cf3b599a6...
docs/build_manual.py	a7006a8ec85dc39aef3e6607fbca1e7c...
docs/figures/architecture.png	17cfffdd3c6c14db7e59189dd20777141...
docs/figures/distribution.png	75d0bdd683208d2c58318e056c31900a...
docs/figures/energy.png	07ba9a5aa93f57cf1fdb611558f14eea...
docs/figures/healthy.png	573353f3544e7d7dfa4e5076d762237b...
docs/figures/recovery.png	d2b0e7047e073ac1eba53a90cb4c88e1...
docs/figures/sensitivity.png	1b536b10d92ad13d4f44302695282aa9...
docs/figures/violations.png	851b8fa847d72a4c142a554819dda713...
fixtures/current_500_expected.csv	dedda9c8f2a74c9b32d234b155b2933b...
fixtures/current_500_inputs.csv	b6d5bc530002d80805d9304063419883...
fleet/__init__.py	e3b0c44298fc1c149afb4c8996fb924...
fleet/benchmark.py	54ee234b1b80c1408404bfcae6c62b80...
fleet/capture/kube_capture.sh	9604384df758ab50f6f133da01823df5...
fleet/capture_replay.py	ecf2b5c608be255edb77eaf7f3da15d...
fleet/harness.py	d216e3ada74d7c2352544e0320d4118c...
fleet/planetlab.py	a13b32a6cd12e8fb4a526bd80e0b4518...
fleet/sim.py	fcda38f76948b3e35a569568775e8c...
fleet/traces/PROVENANCE.json	62629c0ce60fac76b22629ad1eec689b...
fleet/traces/planetlab/20110303_146-179_surfsnel_dsl_internl_net_root	6fbc3e6a34e328c4991cb5bf2b4d66ba...
fleet/traces/planetlab/20110303_75-130-96-12_static_oxfr_ma_charte_r_com_irisaple_wup	d169ac6090c88481e15ea5af63fbaa05...
fleet/traces/planetlab/20110303_75-130-96-12_static_oxfr_ma_charte_r_com_root	3ec2aa408473cb327d84c18bd987f1d4...
fleet/traces/planetlab/20110303_ait05_us_es_irisaple_wup	05afb88829c33b2351559f9b4de5f0f3...
fleet/traces/planetlab/20110303_ait05_us_es_princeton_codeen	5923c0b4f7bcc4b32e1140c71abfad3c...
fleet/traces/planetlab/20110303_ait05_us_es_root	694f4f59a4d900433b86d90367f4798e...
fleet/traces/planetlab/20110303_chimay_infonet_fundp_ac_be_irisaple_HEAP	c411767f05dbb657762c7de677ffe64d...
fleet/traces/planetlab/20110303_chronos_disy_inf_uni-konstanz_de_root	69b3e564a5615b1838a1ced7300b2d79...
fleet/traces/planetlab/20110306_75-130-96-12_static_oxfr_ma_charte_r_com_irisaple_wup	76999499112a94d028f2f52cf925bba7...
k8s_fleet/__init__.py	e3b0c44298fc1c149afb4c8996fb924...
k8s_fleet/benchmark.py	5be462ac23bff1ae5187498f2f9286db...
k8s_fleet/controllers.py	ed03d4bdc2003bb8ed88d8e3145189bb...
k8s_fleet/plant.py	791a8c16c853300c8566f4fe19129ed8...
omni_controller/__init__.py	e3b0c44298fc1c149afb4c8996fb924...

File	SHA-256
omni_controller/controller.py	52de3cb60ca3f53b7290c62dad46d86...
omnicompass/__init__.py	e3b0c44298fc1c149afb4c8996fb924...
omnicompass/adapters.py	57ca59da4024ccea2769083998fdeb7b...
omnicompass/core.py	bd615f156169f6794b41439c4ce0726d...
omnicompass/muscles.py	8aa5e61437f59a4d3ffbdd2c810b43b1...
omnicompass/shield.py	38ca47da32ebdd935e44cfe712d79cb...
omnicompass/stack_sim.py	ac83780fde0423a5b8f014cac7b9e417...
pilot/__init__.py	e3b0c44298fc1c149afb4c8996fb924...
pilot/score.py	3b8b15667abc8dcf6fa7d5bc95dda871...
pilot/selfpilot.py	a90e0a6515479d275c68dc657a790fdc...
reference/PROVENANCE.json	cb72f8ca47610ee1a01c8e2c6016ff74...
reference/omni_compass_reference_engine.py	92799376dd9a626cef330d15f3fc3603...
requirements.txt	d9d0c7f3ba8c3558319ff359557f65c4...
results/MULTIPLICITY.json	e3d600ff671d345fd58a498227b18a9a...
results/PREREGISTRATION.json	a659e6a5f81dcf6c8c26ead7b8f914ba...
results/SAVINGS.csv	b76edbf58b780cb17e0c980def31acbe...
results/SAVINGS_INPUTS.csv	30257b935d458f1acb682b26a478cc52...
results/SOAK.json	5580668710b41eb2495df7eb10d1c657...
results/VERIFY_LOG.txt	3ea47e893ea107abada0b9f12330511e...
results/core_evidence.json	cfc054c5c4c664f8b8bfd85254f975e4...
results/development/JOINT_121.jsonl	d5c214b8e99c58fb293bff9779fca064...
results/development/REFINE_61.jsonl	2db033583d20ec66fa9661973afe9d9d...
results/development/SEARCH_81.jsonl	020125300f9cf743e045a9050a139bb6...
results/development/joint_search.py	06e252cac2aad6fde98e00a8f652b460...
results/development/refine.py	a0a4e319b1c3e88adf568dbebca41ab8...
results/development/search.py	c265d0fce79eafadce16f5aaa10fbe24...
results/development/seed_1000/RUNS.csv	fc029fc484bf464894d1d4699ed44242...
results/development/seed_1000/SUMMARY.json	7242e45a1b4985694fa9a607fa3c43a3...
results/development/throughput_search_round1.jsonl	6b3f5d0106f3f47c34f8bf72f0262c33...
results/development/throughput_search_round2.jsonl	8fa3c6321494b6dfa679b056b076300d...
results/fleet/PREREGISTRATION.json	0d039f3ba9600d67542ecc0eea3a80f5...
results/fleet/development/SEARCH_70.jsonl	0fab478a5fd27685779f99323de39a80...
results/fleet/development/SEARCH_90.jsonl	b391f0222794422844e28dec5d5c2ea3...
results/fleet/development/search_70.py	08a274907e5f78cdb804576075aa817e...
results/fleet/development/search_90.py	00fc785d1da2547d7af39ae5f86670fb...
results/fleet/heldout/RUNS.csv	bcd48bc2c5bdf1b6850a2ddb9453515...
results/fleet/heldout/SUMMARY.json	eb6349bc939db10275e1578cc8506407...
results/fleet/planetlab/RUNS.csv	d71061b565914f47e3ee341afe4f3795...
results/fleet/planetlab/SUMMARY.json	0d4ddef5b50995cd51f9d99e6e893253...

File	SHA-256
results/fleet_overhead.json	8670adb0bd33890dfd47535419a5a8e6...
results/heldout_seed_346410161/RUNS.csv	70ffe85d33bb23379c5c2a23fe57c2de...
results/heldout_seed_346410161/SUMMARY.json	d1b1367a162e208fa9347a9447348748...
results/heldout_seed_360555127/RUNS.csv	bf2cca7045aadcf06530f0442f6ab4b...
results/heldout_seed_360555127/SUMMARY.json	b376e8d0087f063bba429b88d32fb09b...
tests/__init__.py	e3b0c44298fc1c149afb4c8996fb924...
tests/fake_cluster/kubectrl	6d559b454656f55b419f8e7c43374613...
tests/test_core_parity.py	3542e07fed5c7b2188244067e5d71a72...
tests/test_cpp_governor_parity.py	51fa282c75fa5ae02af1fc99de32242e...
tests/test_cpp_hpa_parity.py	840bb301f602b9801d3b184c461aa922...
tests/test_cpp_shield_parity.py	c6bb677c9d95fef47f87082200c87d75...
tests/test_fleet_reald_data_paths.py	1b7348f53430b55c097d50c46b5f5d0a...
tests/test_hpa_three_way.py	ce2c58a592c331b418f9abbac1605146...
tests/test_native_equivalence.py	9f5707aff32332bdac7ffb52f85c7b56...
tests/test_omni_controller.py	f3bad3f384cd0ba88259e666fe1c4f77...
tests/test_pilot_score.py	471fc1c8679a0ceb5b3c5ff0e295b565...
tests/test_selfpilot.py	d489aa7a37bd20ead1b2e39ccf05d841...
tests/test_stack_sim_provenance.py	6cf536e4002b38248912e2da0d1e1cc4...
tests/third_party/README.md	7d4ec0c8a09856293e355ed4490f569e...
tests/third_party/hpa_independent.py	e5f79dbdc88c9c50dbf028c7c2fd9ca7...
tools/code_fingerprint.py	abb0e8a27c8f11d46a6e10c7f90213e2...
tools/strip_reference.py	1dbab6494b976ec7f9e934b53fca9716...
verify.py	f78f64c6d9723c6c00a8785d136feec2...